T8- EXCEPCIONES EN JAVA

Sesión 1:

Las excepciones son **MECANISMOS de programación** para **DETECTAR y CONTROLAR ERRORES** de software.

GESTIÓN DE EXCEPCIONES: conjunto de **técnicas que un lenguaje de programación** proporciona para **DETECTAR y MANEJAR los errores** que se puede producir **EN TIEMPO DE EJECUCIÓN**.

Esto significa que las excepciones no detectan errores de compilación. Para eso están otras herramientas como los IDE o los propios compiladores.

Hay **dos formas** de gestionar los **errores en tiempo de ejecución**:

1. **Enfoque de programación estructurada**: los métodos se diseñan para que **si ocurre un error DEVUELVAN un valor erróneo** o nulo. Sin embargo, cuando ocurren errores que provocan la finalización del programa (error fatal) esta técnica es insuficiente, porque **no permite conocer la CASCADA de llamadas entre métodos**.

2. Los **lenguajes "modernos"** como C++, Visual Basic o Java (principalmente los **lenguajes orientados a objetos**) han introducido la noción de **EXCEPCIÓN**, que es una **técnica de PROPAGACIÓN DEL ERROR entre métodos** y que sí **permite conocer la CASCADA de llamadas** entre métodos cuando se produce un error fatal. Así, en aplicaciones actuales que

tienen muchas capas de software superpuestas (BBDD, interfaz gráfica, capa de red, etc.) cuando se produce un error podemos saber dónde se ha producido.

1. TIPOS DE EXCEPCIONES EN JAVA

Hay diferentes **tipos de excepciones**:

* **RuntimeException**: errores de **PROGRAMACIÓN** que deben ser controlados durante el proceso de DESARROLLO de software. Por tanto, no son responsabilidad del lenguaje sino del programador. El objetivo con este tipo de excepciones es PROGRAMAR para EVITAR que OCURRAN, porque se pueden evitar.

Por ejemplo, son de tipo RuntimeException

- ArrayIndexOutOfBoundsException,
- ArithmeticException,
- etc.

* **Excepciones obligatorias**: todas las excepciones que Java no considera de tipo RuntimeException, y que se producen por **motivos que el programador no puede controlar**. **Java OBLIGA a gestionar** todas estas excepciones. Es decir, cada vez que escribimos una línea de código que puede provocar excepciones de este tipo, Java nos obligará a tenerlas en cuenta.

* **Excepciones de usuario**: como programadores, podemos **escribir nuestras propias excepciones y lanzarlas** cuando creamos necesario.

Sesión 2:

2. GESTIÓN DE EXCEPCIONES

Se pueden GESTIONAR las excepciones de dos formas: **CAPTURA o DELEGACIÓN**.

* **CAPTURA** de excepciones: se utilizan **bloques try-catch-finally**.

1. Java **INTENTA (try) ejecutar** el código que está dentro del bloque try, que es **código que puede lanzar excepciones obligatorias**.
2. Puede haber **uno o más bloques catch**. **Cada catch captura una excepción diferente**. Cuando se produce una excepción, la ejecución salta al bloque catch correspondiente y ejecuta lo que hay dentro. Si hay varios catch que coinciden con el error, se ejecuta el primero de ellos. Es decir, el orden en el cual están escritos los catch es importante.
3. El bloque **finally, que es opcional, contiene código que se ejecuta SIEMPRE** y al final, tanto si se produce una excepción como si no. Normalmente se utiliza para finalizar correctamente el programa: cerrar flujos de datos, dar mensajes, etc.

* **DELEGACIÓN**: un **método lanza excepciones** poniendo en su **cabecera throws** y la lista de excepciones que lanza

```
public float dividir(int x, int y) throws ArithmeticException{

    return x/y;
```

}

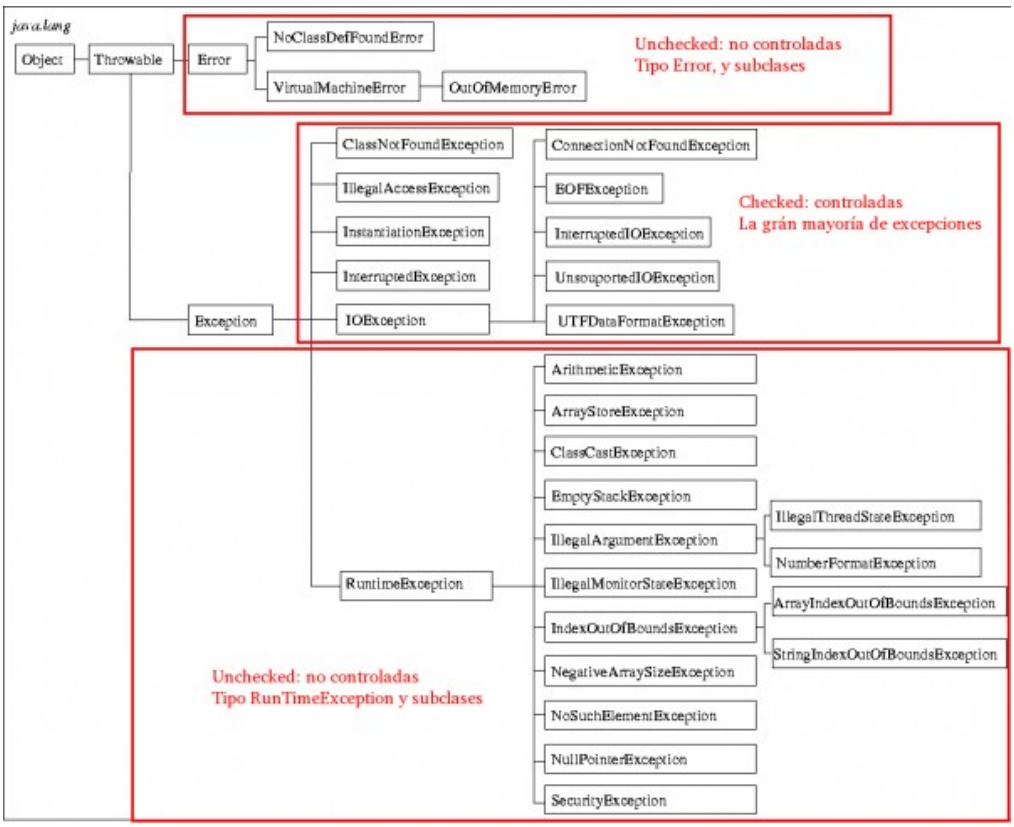
Cuando un método lanza una excepción, la está propagando por la pila de métodos en ejecución. **Alguno de los métodos superiores tendrá que capturarla y gestionarla** con try-catch.

3. EXCEPCIONES COMO OBJETOS JAVA

En Java **las excepciones son OBJETOS**, que tienen sus correspondientes métodos.

Como objetos que son, la excepciones Java están organizadas en jerarquías de herencia. Por ejemplo, ArithmeticException es descendiente de RuntimeException y por eso no es obligatorio gestionarla.

La clase Exception es la raíz de todas las excepciones de Java.



La **jerarquía de excepciones es importante** a la hora de poner múltiples catch en un mismo bloque try, **porque si hay varios catch** y se produce una excepción, la **ejecución de catch va en el orden en que están escritos**. Por este motivo, las excepciones más generales (antepasados) deben ponerse al final de la lista de catch y las más específicas (descendientes).

Por otra parte, la jerarquía de excepciones permite agrupar varias excepciones cuando no es necesario ser muy específico.

4. EXCEPCIONES PROPIAS

Como programadores, **podemos crear** nuestras propias excepciones **EXTENDIENDO de las clases de Java**, teniendo en cuenta que si extendemos de Exception será una excepción de

gestión obligatoria, mientras que si extendemos de RuntimeException será de gestión opcional.

Para lanzar una excepción propia es necesario

* instanciarla
* lanzarla con throw.

```
public class ZeroDivisionException extends ArithmeticException{
...
}

ZeroDivisionException e= new ZeroDivisionException();
throw(e);
```